

# 创始人手册： 打造 AI 原生 创业公司

# 目录

|                  |    |
|------------------|----|
| 为 2026 重启的创业生命周期 | 3  |
| 创始人的意义正在改变       | 5  |
| 构想阶段             | 8  |
| MVP 阶段           | 15 |
| 发布阶段             | 21 |
| 规模化阶段            | 25 |
| 工作没变，规则已新        | 31 |
| 资源               | 33 |

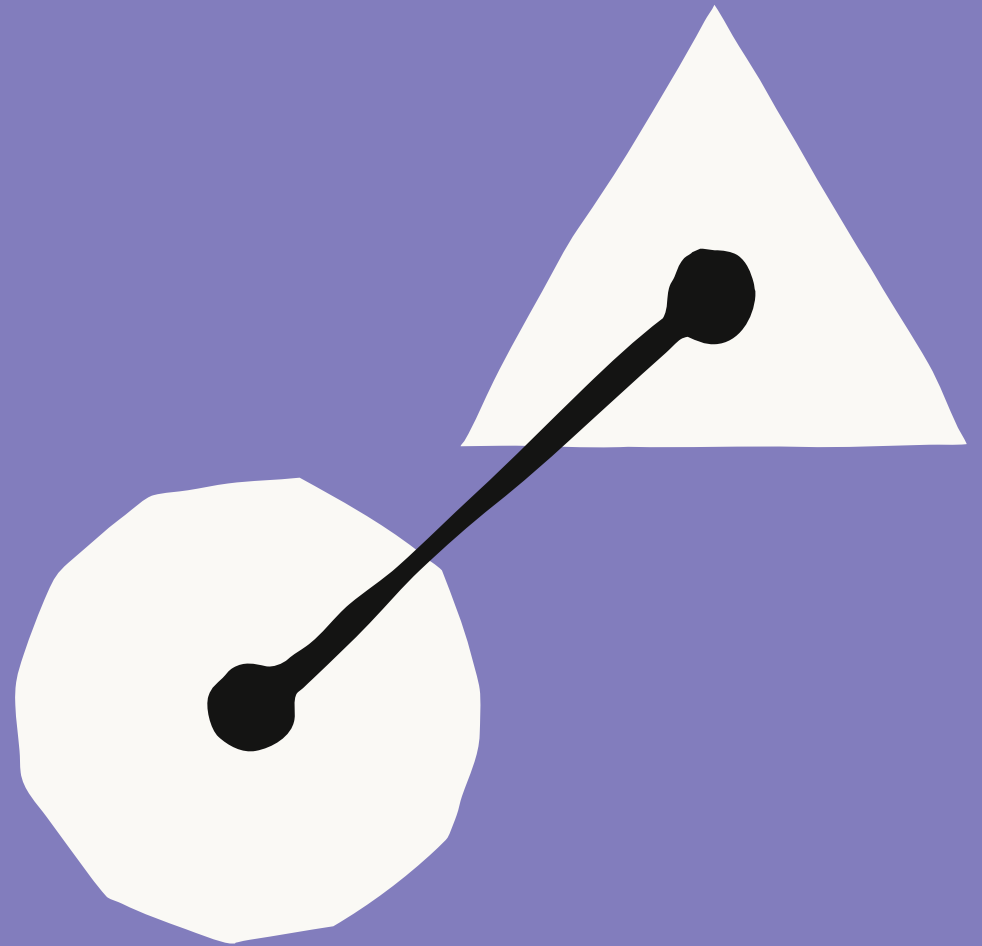


第 1 章

# 为 2026 重启的 创业生命周期

# 为 2026 重启的创业生命周期

AI 正在重塑创业公司的构建方式。从未写过一行代码的创始人，如今也能交付生产级应用；而精简的十人独角兽，已从草根逆袭的故事变成深思熟虑的行动方案。在 2026 年，AI 可以编写生产代码、开展市场调研、综合分析竞争格局、起草投资材料，并自动化运营流程。即便是经验丰富的技术型创始人，过去在整合实现创意所需的工具、平台与系统时也要面对陡峭的学习曲线；AI 消除了这些曲线，最重要的是，它把“谁能创办公司或打造产品”这件事的门槛拉平了。在 2026 年，一个好点子能让创始人比以往走得更远。智能体编码把过去需要一支工程师团队才能完成的工作，压缩成创始人自己就能交付的事。传统的创业增长曲线假设：从创意到规模化的路径是验证 → 融资 → 招人 → 构建 → 再融资 → 增长 → 招更多人 → 循环往复。如今，AI 已经抹去了“生命周期每进入新阶段就需要更大团队、不同技能组合和一轮新融资”的预期。本手册依据这些新现实，重新绘制创业旅程的四个核心阶段（构想、MVP、发布与规模化）。我们考察当 AI 成为你技术与组织发展的核心时，每个阶段会是什么样子，每个阶段的合适工具是什么，以及运用这些工具的创始人如何压缩时间线。如果你已准备好规划出从创意到退出之间的最短路径，请继续读下去。



## 第 2 章

# 创始人的意义 正在改变

# 创始人的意义正在改变

过去，创始人由他们能做什么来定义：技术型创始人写代码，非技术型创始人负责业务运营和成交。但 2026 年创始人可以使用的模型、系统和 AI 智能体，已经消解了“能构建的人”与“拥有值得构建的想法的人”之间的那堵墙。AI 原生创业公司正在从根本上改变创始人的含义。如今，没有工程背景的人也能构建出让创意落地的生产级软件；而技术娴熟但缺乏商业知识的创始人，也能轻松产出市场进入策略、财务模型和高度精良的融资演示。历史上，创始人把大部分时间花在执行模式上：写代码、管人、处理日常运营。在 AI 原生创业公司中，创始人的角色更少是个人贡献者，更多是智能体的编排者——那些能读取文件、运行命令、执行代码、甚至浏览网页的专用 AI 助手。创始人的注意力沿着层级上移，转向更高阶的工作：产生创意，并指挥那些把创意落实的系统（AI 智能体、工具，以及任何存在的小团队）。然而，AI 作为核心基础设施最具革命性的结果，是为拥有专业领域知识的非技术型创始人扫清障碍。当创始人的来源不再局限于有工程背景的人，你就会看到由拥有截然不同人生阅历的人创办的公司，去解决传统技术创始人通道从未优先（甚至从未注意到）的真实问题。

## 面向精简创业公司的 AI 工具能力

传统创业模式假设：你需要招工程师来构建、招销售来卖货、招运营来运转业务。人头数被视为组织势能和产品成熟度的标志。

2026 年的早期创业公司则截然不同。它们在设计上就极度精简，往往只有创始人一人，或加上寥寥数人。通过把技术与组织发展都围绕“AI 即基础设施”来构建，它们可以在扩张团队之前就达到产品验证、早期营收，甚至盈利。AI 尤其在三个领域帮助创业公司像一个大得多的组织那样运转：研究、智能体编码，以及关键业务运营的工作流自动化。

## 对话式智能与研究

可以这样理解：每个领域随叫随到的专家。想想一位创始人在第一年里几乎必然需要知道、却几乎必然不懂的所有事情：我怎么设置薪酬发放？我怎么规划产品开发冲刺？我怎么起草一份精炼的投资人备忘录？这类早期创业问题过去的答案都一样——“找个懂的人”。对自筹资金或种子前的创始人来说，这要么消耗本该用于构建的时间去搜集知识，要么需要烧掉一笔早期资金去请顾问。如今，他们把 AI 当作各种可想象领域随叫随到的专家。

- 深度研究：竞争分析、市场规模测算、财务建模
- 文档起草：融资演示、案例研究、投资人备忘录、PRD
- 战略思考伙伴：唱反调式分析、事前验尸、情景规划、路线图优化

## 智能体编码

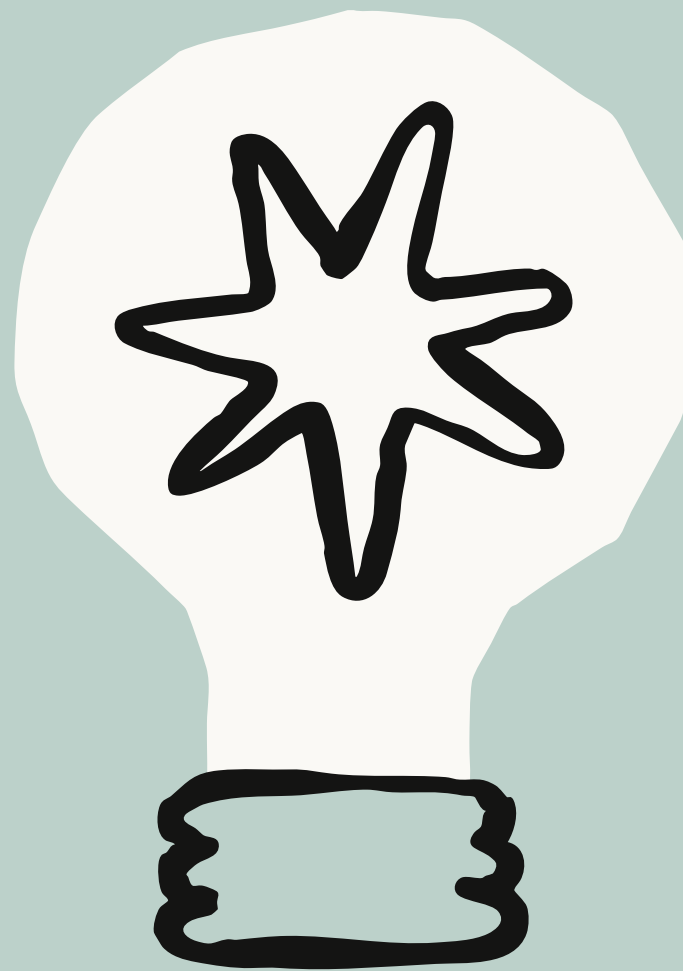
可以这样理解：那位永远在线、从不被卡住的工程师。过去，构建软件需要一位技术联合创始人、一家外包开发团队，或一段足够长的资金跑道来招一支工程团队，然后你才能写下第一行生产代码。如今，智能体编码工具让每一位有志创始人都能用自然语言描述想构建什么，并指挥 AI 以一支完整工程团队的速度和规模去生成、测试、调试并重构生产级代码库。从“我有一个想法”到“我有一个产品”的时间线已被压缩。如今创始人的角色聚焦于构建什么以及为什么构建，而 AI 负责实际搭建面向真实用户、可投入使用的基础设施。

## 工作流自动化

可以这样理解：按需调用、自动化的运营团队。即便创始人能像顾问一样做研究、像工程团队一样构建，仍有一整类工作——超出战略规划或产品开发之外——必须有人完成。排期、更新 CRM、拉取周报、保持文档最新、发布内容、跟踪合规要求、维护公司赖以运转的各种工具与系统之间的连接组织，这些也都得做。在精简的创业公司里，这些负担主要落在创始人身上——而它对本应投入更高阶决策的时间和注意力，是一项沉重的税。借助 AI 工具的工作流自动化可以卸下这项税。重复性的运营任务可以配置为自动发生：交易推进时 CRM 自动更新，周报自我汇编，产品文档随产品变更同步更新。而且，关键在于，Claude Cowork 能与创业公司赖以运转的互联系统集成——你的项目管理工具、你的沟通栈、你的数据源——而无需有人去构建并维护这些集成。在“第零天”的创业公司里，那个人几乎总是创始人。

## 时机与编排就是一切

能有效驾驭 AI 研究、自动化与智能体编码能力的创始人，可以打造出一家运转杠杆远超其人头数的创业公司。他们也得以把绝大部分时间和精力投入到真正重要的工作上。这项工作并非自动驾驶；编排这些 AI 工具的创始人需要知道如何（以及何时）运用它们。本手册余下的部分专门探讨创始人沿着 AI 原生创业之路前进时会遇到的目标与挑战，以及如何在旅程的每个阶段有效运用 AI 工具。



第 3 章

# 构想阶段



# 构想阶段

每位创业创始人都从同一个地方出发：一个让他们念念不忘的问题。这是创意与现实相遇的创业阶段：在 2026 年取得创业成功，需要一种纪律——在证据足以支撑之前，不要构建。这一阶段的工作是研究、客户探索、竞争分析，以及对反面证据的诚实评估，全部都要在请 Claude Code 生成你第一行生产代码之前完成。

## 构想阶段目标

在构想阶段，创始人的主要目标是以研究为导向的验证：在投入资源构建之前，搜集扎实的证据，证明一个真实的问题确实存在（并且你提出的方案能有效解决它）。实际操作上，构想阶段就是创始人需要大致按以下顺序回答的一系列问题：

- 这个问题是否真实、具体，且频繁到值得围绕它构建？
- 究竟谁有这个问题，这是否构成一个市场？
- 是否有其他人在解决它？如果有，方式如何、做得多好？
- 要解决这个问题，一个方案究竟需要做到什么，而我的想法做到了吗？这些追问的结果汇总起来，回答一个单一的终极问题：这值得构建吗？这意味着在行动之前先做到具体。“人们在报销上很头疼”是一个观察。“中型企业的财务经理每周花四个多小时核对提交，因为他们现有的工具无法与其会计软件集成”才是一个可检验的假设。

## 构想阶段退出标准

构想阶段的退出条件是找到问题—方案契合。你已经建立起定性证据（主要来自与真实人类的对话），证明在开始构建解决方案之前，你确实是在为真实的人解决一个真实的问题。当你能对以下三点全部回答“是”时，你就准备好离开构想阶段了：

1. 问题是否真实且具体？在这里给出肯定回答，要求你能准确说出谁经历这个问题、他们多久遇到一次、问题对他们影响多严重，以及他们目前是怎么应对的。
2. 你的方案是否解决了真正的问题？不是你最初假设的那个问题，而是验证过程揭示出来的那个。有时两者相同，但并不总是如此。
3. 你是否有足够的信号来支撑构建？在这个阶段你永远不会有确定性，等待确定性本身就是一种失败模式，但你需要足够的定性证据，证明投入做 MVP 是一个经过推理的决定，而非一种信念之举。

## 构想阶段的挑战

构想阶段是你创业旅程中最重要的工作发生之处，因为这里也是后果最严重的错误发生之处：现在搞错某件事，会很快让你刚萌芽的事业彻底脱轨。不过，构想期的多数挑战都涉及“前进速度快过你的理解所能支撑的程度”，所以审慎而深思熟虑地推进的创始人，会获得稳健的进展。

## 把构建误当作验证

挑战：当技术障碍被移除，一位满腔热情的创始人就有可能跳过创业旅程中最重要的工作：验证他们的想法是否真的是人们需要且会使用的方案。即便在当前智能体编码时代之前，也有 42% 的创业公司因为构建了没人想要的东西而失败。而如今，像 Claude Code 这样的智能体编码方案，已经把“我有一个想法”和“我有一个产品”之间的距离急剧压缩，那个失败率只会继续攀升。虽然对一个绝妙好点子的创始人来说，从未有过比现在更好的时代，但能如此迅速、轻松地拼出一个看起来像产品的原型，反直觉地，也给 AI 原生创业公司带来一种真正危险的存亡风险。直到不久以前，构建还需要真实的开发时间和预算，连拼出一个基础原型通常都要几个月。如今技术开发的门槛在很大程度上消失了，AI 让创始人极易直接跳进构建，而没有在真实世界中验证其效用。达到问题—方案契合，要求先验证你的假设，然后再构建；但许多首次（甚至有经验的）创始人误以为 AI 能绕过这一要求，把流程变成：有一个想法 → 立刻构建一个原型 → 把原型的存在当作验证。原型成了相信“假设一开始就对”的理由，却从未检验它是否真的成立。一个能用的原型很容易被误当作“你在解决真实问题”的确凿证据，但它不是。你的原型应当充当与潜在用户对话时一个有用的压力测试道具。这些对话本身才是真正的证据。

## 过早扩张

挑战：当构建毫不费力且即时，你就会把执行扩张到远远超过业务需求的程度。过早扩张意味着在你真正验证某条产品路径值得投入之前，就已经投入了它。这一直是创业杀手，但 AI 让创始人在不知不觉中掉进过早扩张陷阱变得容易得多。智能体编码助手如此强大，以至于很容易在没有刻意决定偏离航线的情况下，就把执行扩张到远超问题—方案契合验证的程度。它会围绕一个根本上有缺陷的前提去生成、测试、调试并重构代码库，热情程度和它对待一个绝妙想法时一模一样。系统里的智慧是你的。这个阶段的最高指令是让你的理解走在你的构建之前——尤其是当构建如此迅速、感觉如此不费力时。

## 客观性的丧失

挑战：向一个 AI 工具索要支持你既有信念的证据，它就会找到。如今的确认偏误，自带一台研究引擎。确认偏误一直是创业中的职业风险：创始人天生就对自己的想法充满热情。如今，AI 工具给确认偏误来了一次重大升级。让 AI 验证你的创业想法，它会找到支持性证据；让它测算你的潜在市场规模，它会找到让你的 TAM 看起来值得投资的那个数字。AI 听从你的指挥，这意味着一个不问尖锐问题的创始人，如今能比以往任何时候都更快地为一个糟糕的想法构建出一套精心打磨、看起来研究充分的论证，同时还满怀信心地以为自己确实在做尽职调查。解药是同一个工具，只是指向相反的方向：AI 压力测试一个想法，会和它验证一个想法一样彻底。当研究和结构化的对抗性思考揭示出你的想法需要修正的证据时，这就是转向的信号。

# Claude 如何帮助构想阶段的创始人

推动你的 AI 原生创业构想穿过构想阶段，可能感觉没完没了。你是创始人，你就想动手构建。但这个至关重要的启动阶段，本质上是一项研究与验证工作，这意味着要去拿那些帮助你在全力投入写代码之前更严谨地思考的工具。下面是在 Claude 的各个产品形态（Chat、Claude Cowork 与 Claude Code）上的使用方式，帮助你在做好充分尽职调查的同时，尽可能快地穿过构想阶段。

Chat、Claude Cowork，还是 Claude Code：  
选择正确的 Claude 形态

AI 让创业创始人更容易更快交付、自动化繁琐 workflow 并规模化运转，但你使用的形态很重要。下面是根据手头任务，何时使用 Chat、Claude Cowork 或 Claude Code。Chat 用于不离开你当前所在应用的快速交流。把它用于经营公司中持续不断的小任务：从一份密集的投资人备忘录里提炼一句话要点、在董事会前对某个说法做合理性核查，或理清与团队的一长串 Slack 讨论。Claude Cowork 用于真正耗时的知识工作：从多个来源汲取信息、加以理解，并产出一份成品，比如一份文档、演示或电子表格。可以想象把一文件夹的客户通话记录变成下次产品评审用的主题化发现文档、在融资前从十几个供应商网站构建竞争格局，或一项每周一早上的常设任务——从你连接的工具里拉取指标，并把每周 KPI 简报放进共享文件夹。Claude Code 是面向你团队中工程师的智能体编码环境：直接访问代码库、Plan 模式、git 集成，以及本地、IDE 或沙箱云环境。在这里，一支精简团队能在不断增长的代码库上交付功能、迁移 MVP 时期的遗留代码，并在不等待更多人手的情况下从原型走向生产。

| 如果任务是…… / 该用 / 为什么                                                                                                                                                         |  |  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|--|
| 一个问题、一次改写、一次快速头脑风暴 → Chat：快速、对话式、无需配置。研究、分析，或基于你的文件与系统构建的成品文档 → Claude Cowork：文件夹访问、连接器、技能、定时运行。编写、测试或交付软件 → Claude Code：代码库访问、diff、git、开发环境。三者底层是同一个 Claude；改变的是它周围的工作空间。 |  |  |
|                                                                                                                                                                            |  |  |
|                                                                                                                                                                            |  |  |

## 定义并压力测试问题假设

你自身的领域专长和前期研究已经产生了一个假设。第一项工作是把它磨锐，直到它真正可检验。Claude 在这里尤其有用，因为它能逼出具体性：究竟谁有这个问题，多频繁，多严重，他们目前怎么应对？一个无法精确回答这些问题的问题陈述，还没准备好去验证。

- 练习：与 Claude 一起打磨你的问题陈述，直到它成为一个可检验的假设。例如，“合同审查耗时太久”并不能有意义地检验。但“中型企业的内部法务团队每个合同审查周期要花 3 天以上，因为红线修改是在邮件往来中管理，而不是在单一的版本受控文档中”就非常可检验。你的下一步是请 Claude 反驳你的想法，并寻找能驳倒你假设的反面证据。这能揭示负面市场信号、失败的竞争者、客户行为模式，以及一份支持性的综述会悄悄降权的结构性障碍。

目标是带着“已经用现有最强反方论点压力测试过自己的假设”的状态进入客户探索，从而让信息性的用户访谈真正开放，而不是一场寻找确认的搜索。注意：把 Claude 当作结构化的“唱反调者”，是 AI 创业生命周期每个阶段的核心用例。

## 市场研究与竞争格局梳理

掂量你的竞争对手。有一种创业公司特有的现象，叫竞争对手忽视：过于专注自己的愿景和执行，以致系统性地低估同一空间里其他人在做什么。幸运的是，AI 提供了解药：请 Claude 给出最有说服力的论证，说明这个方案空间里的某个竞争对手为何会成功、而你为何不会。Claude 可以分析为何他们的做法实际上更好、为何客户会选择他们、为何你潜在的差异化点可能没有你以为的那么有护城河。

- 练习：请 Claude 按层级梳理你的竞争格局：直接竞争者、间接竞争者、潜在收购方，以及可能切入你领域的相邻玩家。然后请它论证为何每一层都对你的成功构成真实威胁——不是那种最容易被打发掉的威胁版本。市场研究：Claude Code 可以综合公开可得的客户反馈，揭示反复出现的抱怨和未被满足的需求。额外好处：这本质上是对竞争对手客户的免费定性研究。

- 练习：指挥 Claude Cowork 跨你的关键来源综合竞争对手评价，找出现有方案尚未解决的头部抱怨。如果你的假设解决了其中一个或多个，那就是问题—方案契合的有力证据。如果没有，这同样值得知道。

Claude Cowork 还能从密集的行业报告、分析师文件和市场研究文档中提取相关信息和数字；接着，这些干净、综合后的输入就成为 Claude 分析工作的理想上下文。

- 练习：用公开数据构建 TAM/SAM/SOM 模型，并对其背后的假设做压力测试。判断市场是在扩张、整合还是已成熟；这一背景会影响你对时机和差异化的思考。绘制买方格局：谁掌握预算、谁影响决策，以及这两者是否是同一个人。趋势分析：最后，用 Claude 倾听早期指标，判断你是否在正确的时机进入。追踪那些已经在讨论你这个问题的 subreddit 和领英群组，以及用户描述其困扰时使用的确切措辞。请 Claude 找出曾解决过类似问题的相似市场，并提炼哪些奏效、哪些失败。揭示可能加速或威胁这一机会的监管、技术或人口趋势。

- 练习：请 Claude 找出三个外部趋势——监管、技术或人口——它们可能在未来两年内显著影响你的市场，并评估每一个对你具体假设而言是顺风还是逆风。注意：本节的市场研究与竞争梳理工作不是一次性的练习。你会在 MVP 与发布阶段继续有新发现并演进你的思考，所以每当你的假设演变时，重复这些练习都很重要。

## 规划并设计客户探索

你通过与产品潜在用户交谈所学到的东西，其质量取决于：(1) 你所问问题的质量，以及 (2) 你是否在向正确的人提问。Claude 在开展客户探索方面尤其有帮助，包括该和谁谈、问什么，以及如何理解你听到的内容。

和谁谈：一个精确的目标画像，远比一份长长的联系人名单更有价值，包括最可能切身经历该问题的具体职位、公司类型、团队结构和资历层级。在此基础上，找出这些人实际可触达的地方——他们聚集的社区、活动、领英群组 and Slack 工作区——并基于他们与问题的接近程度，建立一个“先联系谁”的优先级框架。问什么：在目标确定后，用 Claude 来构建访谈框架本身：正确的问题、正确的顺序，并以揭示人们实际做什么（而非他们以为自己会做什么）的方式来组织。新手创始人常犯的错误，是问一个泛泛的、面向未来的开放问题（“你会用类似这样的东西吗？”），而不是具体地追问相关的过去（“跟我讲讲你上一次处理这个问题是什么时候。”）。Claude 也能标记出你草拟的问题里哪些在诱导受访者、过于宽泛，或在其他方面可能产生噪声而非信号。Claude 还能帮你设计追问问题，以探查回避，或就重要问题的含糊回答深挖。如果你的假设涉及不止一个画像，Claude 也能为每个画像设计不同的问题集。财务经理和 CFO 与同一个问题的关系不同，单一访谈框架会抹平这种区别。

- 练习：先手工起草你的访谈问题，再请 Claude 审查它们。具体请它标记出任何具有诱导性、面向未来、过于宽泛，或更可能产生“社会期许”而非诚实回答的问题。然后请它为访谈中最可能产生回避的两三个时刻，建议一个追问。访谈后分析：每次对话后，用 Claude 做复盘：把你的笔记喂给它，请它找出哪些证实了你的假设、哪些挑战了它、哪些是真正出乎意料的。

一旦你收集到一批访谈，把全部访谈笔记跑过 Claude Cowork，揭示反复出现的主题、矛盾，以及双向的最强信号。然后把综合后的输出拿回给 Claude，请它指出你对数据的解读在哪里可能是在向“你想听到的”而非“实际存在的”做模式匹配。

- 练习：每做五次访谈，指挥 Claude Cowork 综合你的笔记并产出两份清单：支持你假设的证据，以及挑战你假设的证据。如果第一份明显比第二份长，请 Claude 判断这种不对称反映的是数据里实际存在的东西——还是你希望找到的东西。

## 客户触达与排期

用 Claude Cowork 自动化围绕“建立联系人名单、开展触达、为用户访谈排期”的运营负担。Claude Cowork 可以使用你与 Claude 共同定义的目标画像（包括职位、公司类型和资历层级），去研究并汇编一份结构化的潜在对象名单和经核实的联系方式。然后它会大规模起草个性化的触达邮件，针对每个人的角色和情境量身定制。当回复进来时，它通过 MCP 连接 Gmail 和 Google Calendar，管理邮件往来、处理排期请求，并把访谈排进日历。工作流继续：Claude Cowork 按既定节奏生成跟进草稿（例如对未回复联系人在第七天跟进），并在每一步完成时更新你的跟踪表，让你始终清楚每个潜在对象处在管道的哪个环节。

- 练习：把你验证过的访谈目标画像交给 Claude Cowork，请它构建潜在对象名单、起草个性化触达序列，并建立一张带有“触达状态、跟进节奏、访谈完成”列的跟踪表。然后让它去跑协调工作，而你专注于为对话本身做准备。

## 设计你的最终方案概念

你已经做完了验证工作：问题真实，你知道谁有这个问题，并且你有一个证据支持的方案概念。用 Claude 从各个角度发展并挑战你的方案概念：有哪些缺口？存在哪些替代方案？要让这个方案在规模上奏效，必须有哪些前提成立？这是一个重要的现实检查点：这个设计究竟解决的是验证过程揭示出的问题，还是你最初进入时假设的问题？

- 练习：把你的方案概念呈现给 Claude，请它找出你的设计最依赖的三个假设。然后问：每个假设要成立，必须有什么为真；如果其中任何一个不成立，后果是什么。

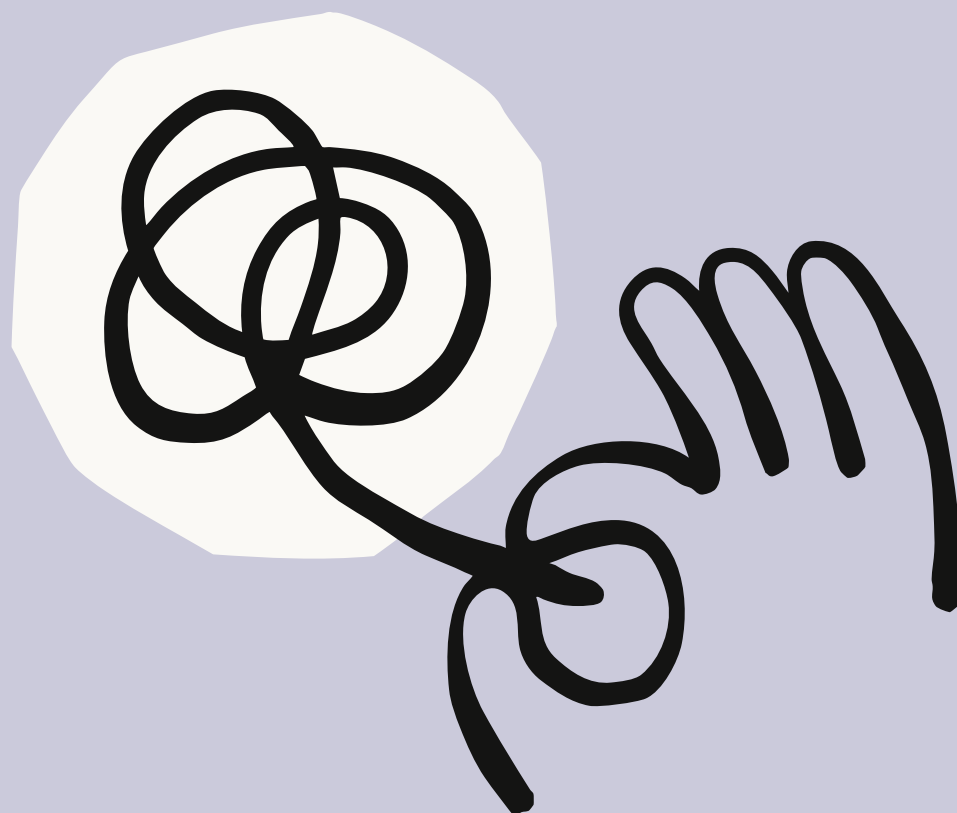
## 用 Claude Code 构建一个轻量原型

现在到了有趣的部分：有了验证过的假设和压力测试过的方案概念，你终于准备好构建点东西了。这是构想阶段中 Claude Code 登场的时刻。即便你一直在零敲碎打地试，现在才是你产出正式轻量原型的节点：把你的想法摆到一个真实的人面前并获得真实反应所需的最小表面积。你（还）不是在构建一个真实世界的产品；你是在搭建一个你想法的可运行样本，用于客户和投资人对话。真实用户对一个他们能真正触摸的东西做出的反应，会告诉你十几次问题——方案探索访谈也问不出来的事。之前，你在确立“你要解决的问题是真实的”；现在，你在请潜在用户与提出的方案互动。

- 练习：定义你的方案所依赖的那个单一核心交互。指挥 Claude Code 只构建那一个。做出来后，把它摆到五个来自你验证过目标画像的人面前，请他们试用。你在这五次对话中学到的东西，决定你是继续构建，还是回到画板从头来。

走到构想阶段的终点，是 AI 创业竞赛中的一次巨大跃进，因为现在你不是在赌一种直觉；你是在依据证据执行。接下来是 MVP 阶段，创始人的指引性问题从“这值得构建吗？”变成“我们究竟该先构建什么？”，而 AI 的主要角色也从研究伙伴转变为施工队。





第 4 章

# MVP 阶段

# MVP 阶段

不少创始人把 MVP 阶段当作施工阶段，但 MVP 阶段本质上仍是一项证据搜集工作。区别在于，你现在搜集的是关于方案（而非问题空间）的证据；具体来说，就是一群真实、可识别的人是否觉得它有价值到愿意使用它、回头用它、为它付费，和/或把它告诉别人。

## MVP 阶段目标

作为一家 AI 原生创业公司的创始人，你的目标是把一个验证过的问题，转化为真实用户真的会用的可运行产品。这不是包含路线图上每个功能的完整版本，而是你想法中最小、最聚焦的迭代——把一个真实方案摆到真实用户面前，并产生产品—市场契合的真实证据。与此同时，你现在如何构建，决定了以后什么可能。这意味着 MVP 阶段还有一个同样重要的第二目标：在不积累那种会复利、并在真实用户大量到来那一刻缠住你的技术债的前提下，快速前进。最后，从第一天起就投资于持久的上下文，是让 AI 保持为力量倍增器、而非熵之源头的关键。在 AI 原生创业公司中，你的代码库是你一次又一次与 AI 协作的对象，这使可读性成为基础。跳过规范、架构决策和上下文文件（如 CLAUDE.md）的创始人，会撞上一堵可预见的墙：每次新会话都要重新解释代码库，而 AI 生成的改动会偏离最初的愿景。

## MVP 阶段退出标准

MVP 阶段的退出条件是产品—市场契合的真实证据：证明一个具体、可识别的用户群体觉得产品有价值到愿意回头使用（留存）、为它付费（营收），或把它告诉别人（推荐）。

## MVP 阶段的挑战

在 MVP 阶段，创始人的最高指令是速度与判断力。这里的挑战都围绕一个核心：你能否足够快地、以正确的方式构建出正确的东西，而不去抄那些日后会让你付出代价的近路。

## 智能体技术债

挑战：因为 AI 基本上移除了过去控制“什么能进生产”的每一个天然瓶颈，速度是有保证的。但当速度是创始人在 MVP 构建中唯一考虑的变量时，他们就有可能积累难以偿还的技术债。MVP 阶段有些技术债是合适的，前提是理解它必须在规模化之前被管理好。它逐渐累积，可以随时间或在一个专门的冲刺中清掉。然而，AI 技术债会复利。如果没有把规范和架构约束写在 AI 能读到的某处，每次会话都会从头重新推导基础决策，而这些决策会漂移。你最终得到一个背后没有连贯心智模型的代码库——不是因为任何单独一块不好，而是因为这些块从未被设计成彼此契合。这是一个真实的问题，而且它确实往往很晚才浮现。



## 误信虚假的产品—市场契合

挑战：AI 工具能产生亮眼的早期数字，但这并不能保证市场需要你的产品。早期势头是创始人能拥有的、心理上最具冲击力的体验之一。在数周或数月的验证工作和审慎、克制的构建之后，交付一个产品感觉像是在确认你一直都对。智能体编码工具能帮你比以往任何时候都更快地抵达这一刻，但早期牵引力不等于产品—市场契合。发布的能量来自转瞬即逝的力量，比如你创始人的朋友、你投资人旗下其他被投公司的潜在买家，或一个带来流量峰值的 Hacker News 头条。不幸的是，这些都无法可靠地预测第六周或第十二周——当那股初始助推消退后——会发生什么。

## 零摩擦的范围蔓延

挑战：当构建感觉毫不费力且几乎免费时，总会有“再加一个酷功能”或“再处理一个边界情况”。这种范围蔓延弊大于利。范围蔓延一直是创业风险。如今的区别在于，对抗它的传统强制函数——工程时间的真实成本——在“加一个功能只需一个下午而非一个冲刺”时，已不再以同样方式存在。这里的挑战是，每一项单独的新增都说得通。产品当然应该处理那个边界情况；用户当然会想要那个工作流。它们在当下并不像范围蔓延，因为用智能体编码每一个都只需极少的努力，但当你的产品蔓延出最初的边界，你就有可能失去方向和势头。解药是在开始构建之前写下一份范围定义，描述产品做什么、刻意不做什么，以及来自真实用户的、什么样的具体证据才足以证明值得新增某个东西。这把决策点从“我们该构建这个吗？”转移到“是否有临界数量的用户告诉我们，没有这个他们就无法从产品中获得价值？”。

## 因经验不足而不安全

挑战：创始人用 AI 工具在不先理解基本安全原则的情况下匆忙把应用推向市场，最终让用户暴露于本可避免的风险中。残酷的事实是，智能体编码工具生成的是能用的代码，而非天生安全的代码。能用的代码很容易，因为功能要么有效要么无效。安全漏洞在被利用之前是不可见的，这意味着没有天然的反馈回路来提醒首次创始人“有什么不对劲”。然而，把一个上线的 MVP 交付给真实用户，意味着真实数据、真实暴露，以及一旦出错的真实后果。轻视安全并非 AI 原生项目才有的新问题。各个时代自筹资金的创业公司，常常把安全考虑拖到构建很晚才做，有时一直拖到生产发布前夕。在任何用户接触你的应用或方案之前做一次安全审查，是把一个最小可行产品推向世界的、负责任的最低门槛。

## Claude 如何帮助 MVP 阶段的创始人

### 在构建之前定义你的架构

在 Claude Code 写下一行生产代码之前，用 Claude 来定义并记录将主宰本阶段所有构建的架构决策：要遵循的模式、要避免的依赖、正在做出的取舍及其原因。这份输出将充当一份聚焦的架构上下文文档，并确立 Claude Code 将在其内运作的护栏。没有这一上下文，每次会话都从零开始，Claude Code 被迫推断它自己的结构性假设。让 Claude Code 在没有护栏的情况下构建，会产出一个功能可用但结构不连贯的代码库；而在不连贯的代码库上迭代和扩张，最终是对时间和 token 的浪费。迟早会到一个点，代码不可避免地崩塌，迫使你从头重建。

- 练习：在打开 Claude Code 之前，先打开 Claude 并描述你在构建什么：它解决的核心问题、它服务的用户，以及你现实预期未来六个月的规模。请它帮你定义应主宰你 MVP 构建的架构原则、在你的约束下应避免的依赖，以及你在这个阶段有意识接受的取舍。接着，把这份输出保存为 CLAUDE.md markdown 文件。这是你的架构上下文文档：你构建的第一个产物，也是之后每一次会话都依赖的那一个。CLAUDE.md 文件充当 Claude Code 的项目级指令，提供项目特定的上下文与指令，当 Agent SDK 在某个目录中运行时会自动读取它们。在功能上，它们是你项目持久的“记忆”。

## 定义并强制执行你的 MVP 范围

无摩擦的范围蔓延是 AI 时代 MVP 标志性的失败模式之一。正如你定义并记录了产品的应用架构，你也需要在任何一个功能被构建之前定义你 MVP 的范围。Claude 能帮你创建一份范围文档，描述你的 MVP 产品做什么、刻意不做什么，以及功能修订标准：在此阶段，来自真实用户的什么样的具体证据才足以证明值得新增某个东西。当新功能想法浮现时（它们一定会），你用 Claude 压力测试它到底是来自用户的真实信号，还是披着产品思维外衣的创始人热情。

## 用 Claude Code 构建你的 MVP

架构和范围一旦定义好，Claude Code 就成为主要的 MVP 构建工具。用它来生成、测试、调试并迭代你的代码库，但把每次会话当作对你已做出的产品决策的执行，而不是抛入一些新决策的机会。

每次 Claude Code 会话开始时，先 (1) 重温你的范围文档，并 (2) 把你的 CLAUDE.md 架构上下文文档提供给模型。每次会话结束时，用本次会话浮现的任何决策更新它。目标是一个你能解释其结构的代码库，而不仅仅是一个能运行的代码库。

- 练习：为你的 Claude Code 工作创建一个简单的会话模板，包含架构上下文文档、本次会话的具体任务，以及要遵守的任何约束或模式。每次会话结束时，向上下文文档追加一条简短日志，详述构建了什么、做了哪些决策、本次会话引入了哪些假设。每次会话五分钟的文档工作，是对“复利成不可管理代码库的架构漂移”的廉价保险。

## 在任何用户接触它之前做安全审查

作为 AI 原生创业公司的创始人，你的责任是知道你代码库里有什么、理解任何潜在的暴露途径，并且不把明显的漏洞交付给信任你、把数据托付给你的真实用户。Claude 可以对 AI 生成的代码做一次有用的首轮安全审查，并能帮助识别常见漏洞。在交付前把它建进流程，是个好习惯。然而它不能替代安全工具，或在更高风险下替代人工审查者——把它当作替代品的创始人，正是那些最终出现在数据泄露报道里的人。Claude Code Security 更进一步：它扫描代码库中的安全漏洞，并为人工审查建议有针对性的补丁，揭示传统方法可能错过的问题。注意：在本电子书出版时，Claude Code Security 处于有限测试版，请在把它建进工作流前确认当前可用性。

- 练习：在部署给任何真实用户之前，用一份具体的简报把你的核心应用代码跑过 Claude：审查认证与会话处理、API 响应中的数据暴露、输入校验与注入风险，以及有已知漏洞的依赖。认真对待每一项发现，评估它是否需要修复，并对任何涉及认证、密钥或数据处理的内容进行人工审查。

## 在发布前构建你的度量框架

那些把早期牵引力误认为产品—市场契合的创始人，通常也正是那些发布后才开始追踪数据的人——而他们选用的指标是为评估“什么在奏效”，而非为揭示“什么没奏效”。解药是在第一个用户出现之前就建立你的度量框架。用 Claude 定义对你这个具体产品而言哪些指标重要、基准是什么，以及数据中什么样的模式构成真正的产品—市场契合、而非讨好你的噪声。具体而言：在发布 MVP 之前设定你的留存基准、激活标准，以及第 7 天和第 30 天目标。接着，定义对你这个具体产品而言“假阳性”长什么样：例如，注册却不激活、有营收却无留存，或最初的热情却无重复使用。当数据到来时，请 Claude 给出针对你自己牵引力的对抗性论证：一个怀疑者会怎么评价这些数字？

## 管理探索与用户反馈的事务

一旦真实用户进入产品，运营层会迅速膨胀。Claude Cowork 处理那些重要但繁琐的工作，比如建立并维护用户联系人名单、运行触达序列、为反馈会安排日程、对 bug 报告分诊，以及追踪迭代周期。在构想阶段管理探索事务的那些 MCP 集成，在这里同样适用。让一个人留在收集回路中，对用户反馈做细致入微的探究。比如，一位用户说“这很棒，但我希望它还能……”，就需要解读：这是核心需求还是锦上添花？是这个客户特有的，还是代表了一个细分群体？缺失的功能是真正的问题，还是上游引导环节出了什么问题？没有工具能回答这些问题。

- 练习：配置 Claude Cowork 来运行你 MVP 阶段的反馈回路：向你的早期用户名单起草触达、为反馈会排期、为 bug 报告和功能请求设计结构化的接收流程，并撰写每周一份的“收到了什么”综合。先自己审阅这份综合；之后，你可以请 Claude 分析这些信息，捕捉你可能忽略的任何重要点。

## 朝证据迭代，而非朝“完成”迭代

MVP 阶段在你拥有产品—市场契合的真实证据时结束，无论产品感觉多“完成”。宣布你已达到产品—市场契合、现在准备从 MVP 阶段进入发布阶段，最终是一项把创始人直觉与所收集证据结合的判断工作。不过，有一些有用的试金石：

- Sean Ellis 测试：问你的活跃用户：“如果你再也不能用这个产品，你会有什么感受？”如果超过 40% 回答“非常失望”，那是一个有意义的 PMF 指标。
- 努力测试：在产品—市场契合之前，留存需要持续干预，包括频繁触达、激励、个人跟进，以及创始人耗费的英雄式精力来维持用户参与。在产品—市场契合之后，产品开始自己做这件工作。当事情开始“拉”而不是“推”时，努力上的这种转变，是“某种真实的东西已经改变”的最清晰信号之一。归根结底，没有任何单一数据点能确认产品—市场契合，因为它是一种必须跨多个迭代周期都成立的模式，你才能确定地这样称呼它。

## 当证据要求时就转向

如果，即便投入了所有这些工作，你就是似乎无法抵达产品—市场契合，怎么办？你的结果不能确认你出发时的方向，这不是失败，而是系统在起作用：MVP 阶段的设计，就是在你对错误答案过度投入之前，把这一信息揭示出来。当数据不支持你当前的产品时，用 Claude 一起梳理这些数据在告诉你什么。

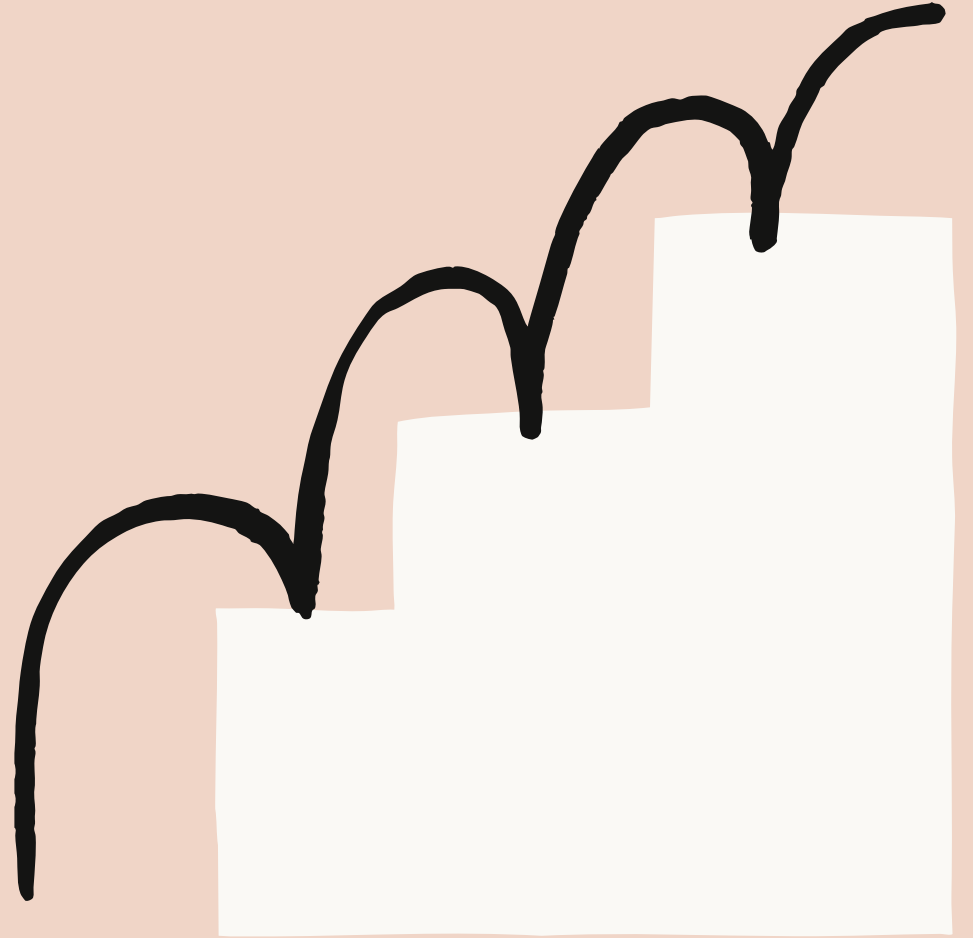
- 探索其他客户细分。也许那些没有转化的用户，从一开始就不是正确的目标。正确的受众往往已经在你的数据里，只是被低估了。
- 调整你产品的价值主张。也许你有正确的受众，但你的 MVP 就是没有引起用户共鸣。对引导、信息传达或核心功能侧重的一次调整，有可能在不改变你已构建内容的情况下解决这个问题。

保持开放，承认这种脱节也许深到需要一次更根本的改变。

- 练习：如果你已完成三个或更多迭代周期，却没有朝你的产品—市场契合基准有意义地移动，在决定下一步做什么之前，用 Claude 跑一次诊断。把你的留存数据、用户反馈和你最初的问题假设喂给它，并问它三个问题：

- 这份数据里是否有某个细分的反应与其余不同？
- 设计价值与体验价值之间的差距，是定位问题还是产品问题？

- 当前产品要找到真正的 PMF，必须有什么为真，而根据你所看到的，那个情景现实吗？让答案决定你是调整、转向，还是回到构想阶段。



第 5 章

# 发布阶段

# 发布阶段

如果说 MVP 阶段是在证明你的产品值得存在，那么发布阶段就是在证明你的业务值得增长。

## 发布阶段目标

在发布阶段，创业创始人必须把早期牵引力变成一台可重复、可持续的增长引擎。除了让产品达到生产就绪，你还必须在加固其底层基础设施的同时，围绕你的产品构建一家真正的公司。在构想和 MVP 阶段，创业公司天然以创始人中心，因为你需要全面的态势感知和紧密的反馈回路。然而现在，仍试图亲自抓住每一条线的创始人，会成为发布阶段的瓶颈。目标不是把你自己从公司里移除，而是构建运营系统，把你的注意力解放出来，去做只有创始人才能做的决策。

## 发布阶段退出标准

发布阶段的退出条件有三个要素：

1. 增长可重复且由渠道驱动。你不只是在留住用户，你是在通过具体渠道、以可理解的单位经济学可预测地获取他们：CAC、LTV 和回本周期都是你知道并能为之辩护的数字。
2. 产品能承受生产负载。基础设施已加固，安全与合规到位，可靠性在真实生产条件下（而不只是你测试过的条件下）经得起考验。

3. 运营无需创始人作为瓶颈即可运转。流程已存在，自动化已到位。你不再是亲自处理支持、分诊、冲刺规划或报告的那个人。

## 发布阶段的挑战

找到产品—市场契合，是早期创业生命周期中最难的问题。如今，创始人的挑战变成了保持它。发布阶段是这样一个地方：找到了真实产品牵引力的公司，如果围绕并支撑产品的组织跟不上，仍可能分崩离析。以下是要警惕的失败模式。

## 技术债到期

挑战：为速度和验证而构建的 MVP 代码库，运行得足够好以证明产品可行，但生产流量、新功能和不断增长的复杂度，如今正把那些近路暴露出来。在 MVP 阶段，积累一些技术债是为速度做出的合理取舍。在发布阶段，那笔债开始产生利息，而且越久不处理，修复就越昂贵。解决办法包括：一次系统性的架构审计以识别结构性弱点、有针对性的重构以解决其中最糟糕的部分，以及一次有意义的测试覆盖扩展，使下一轮功能工作不会重新引入同样的问题。

## 创始人成为瓶颈

挑战：在 MVP 阶段，创始人身处每一个回路是一种资产。在发布阶段，随着支持量增长、产品决策堆积、运营复杂度倍增，同样的本能变成了约束。从“做这项工作”到“设计去做这项工作的系统”的转变，是创业生命周期中最难的转变之一。因为它发生时很少有一个清晰的时刻，风险在于完全错过它，停留在构建者模式，而组织在你周围停滞。这正在发生的明显迹象包括：本该一小时的决策现在要拖一周你才腾出手；支持请求堆积，因为只有你知道答案；以及只有当你亲自记得去做时才会发生的运营任务。补救办法是对你亲自处理的一切——从最微小的任务到最高风险的决策——做一次彻底审计，以识别哪些可以系统化、哪些可以委派，哪些确实仍值得创始人的时间和注意力。

## 安全与合规不再可推迟

挑战：把安全与合规措施保持简单，在 MVP 阶段还行，但现在有了真实用户、真实数据，以及可能摆上桌面的企业合同，它就变成了一种负债。在 MVP 阶段，只有寥寥几个测试用户、生产中没有敏感数据时，安全漏洞是理论风险。然而，一旦你的产品带着依赖它的真实用户进入生产，这种假设就变成了非常真实的暴露风险。此外，不适用于原型的合规要求，在你处理客户数据、处理支付，或销售进入受监管行业的那一刻，绝对适用了。补救办法是在生产规模到来之前（而非之后）做一次系统性的安全与合规审查，并把浮现的一切都当作必须的整改——而不是建议——在下一波用户到来之前完成。

## 在你准备好之前就扩张

挑战：新市场和融资机会看起来像增长机会。它们也可能是产品—市场契合走向消亡的地方。你已建立的初始牵引力是真实的，但它也特定于你的早期受众。过早扩张进入一个与你原市场有显著差异的市场，会引入新的用户行为、合规要求、支付基础设施，以及你产品并非围绕其设计的基线预期。突然之间新变量太多，你失去了清晰解读自身数据的能力。你也冒着这样的风险：在追逐一个新的、未经验证的受众时，忽视你原有的用户群。

## Claude 如何帮助发布阶段的创始人

在发布阶段，Claude 的三种形态全部投入使用，并且彼此支持：每个工具产出的东西都成为另外两个的输入。结果有机地复利，一个把三个工具一起使用的创始人，得到的多于各部分之和。这正是让超精简创业模式在结构上成为可能的原因。当 Claude Code 构建产品、Claude Cowork 围绕它构建公司、而 Claude 帮助把这种产品与组织知识运营化时，一个小团队能像一家规模数倍于它的公司那样运转。

## 在技术债复利之前整改它

你的 MVP 代码库能用，但它也需要一次系统性的整改梳理，寻找任何可能变成结构性负债的技术债。首先，用 Claude Code 跑一次完整的架构审计：识别代码库脆弱之处、任何会变得维护昂贵的近路，以及测试覆盖薄到下一轮功能工作会重新引入同样问题的地方。



把 Claude Code 的审计结果反馈给 Claude，对整改工作进行分诊和排序：什么需要在下次发布前修复、什么可以等一个冲刺、什么在你当前阶段属于可接受的持续性债务。这也是记录你在 MVP 阶段做出的架构决策（那些因为没时间写下来而只活在你脑子里的决策）的时刻。现在把它们写进 CLAUDE.md，能确保未来每一次 Claude Code 会话都从“系统如何设计、为何如此设计”的共同理解出发。

- 练习：指挥 Claude Code 审计你的 MVP 代码库，产出一份按优先级排列的结构性弱点、测试覆盖缺口和重构候选清单。然后把这份清单喂给 Claude，请它把整改工作排布到你接下来的若干个冲刺中：任何你需要先解决的重要问题、可以与功能开发并行处理的事项，以及可以等待的事项。

## 构建替代创始人注意力的系统

构建那些把你注意力解放出来、去处理只有创始人才能应对的职责的运营系统，前提是准确知道你的注意力流向哪里。用 Claude Cowork 对你当前的运营负荷做一次结构化审计，记录每一项重复任务、每一个落到你桌上的决策，以及每一个仅仅因为你亲自记得才会发生的工作流。然后让 Claude Cowork 把这份清单分类为：可以完全自动化的、需要一个人但不一定是你的，以及确实需要创始人判断的。审计完成后，用 Claude Cowork 为自动化候选设计工作流逻辑：什么触发每个工作流、决策规则是什么、输出长什么样，以及完成后去往哪里。

## 把安全与合规变成一条产品工作流

用 Claude Code 揭示在 SOC 2、GDPR 或 HIPAA 审计中频繁出现的代码层面问题，以及你目标市场要求的标准。这会同时揭示漏洞和合规缺口。把这些发现喂给 Claude，帮你把整改工作排定优先级，并设计企业买家在签约前会要求的控制、审计

日志和访问管理。注意：AI 扫描是一种辅助，但不能替代合格的合规审查。接下来，把合规工作流建进你的开发周期，而不是把它当作一次性项目来跑；合规文档需要持续维护和更新。对于正在接洽企业合同或国际市场的创始人，这也是 Claude Code 安全扫描能帮你为一次独立安全评估做准备的时刻。

- 练习：用 Claude Code 跑一次面向你目标市场所要求框架的代码层安全审查。把输出喂给 Claude，请它产出两样东西：一份按优先级排列的安全整改顺序，以及一份你需要产出的文档与控制清单，以满足潜在企业买家的合规审查。

## 搭建你一直在跳过的产品管理流程

发布阶段需要一套轻量、可重复的流程，无需创始人介入即可触发或运转。用 Claude 设计你的产品时间线和工作周期如何组织、在 Claude Code 触碰一个功能前一份规范需要包含什么、bug 报告如何被分诊和路由，以及你的每周指标报告涵盖什么、如何分发。流程设计完成后，用 Claude Cowork 构建并运行运营层：安排冲刺仪式、把进来的 bug 报告路由到正确的地方、从你连接的数据源汇编每周指标，并维护那条让用户信号持续流入产品决策的反馈回路。

- 练习：请 Claude 设计一套轻量的产品管理操作系统：一个明确的冲刺节奏、一份最小规范模板、一棵 bug 分诊决策树，以及一份从你真实数据源拉取的每周指标简报。然后设置 Claude Cowork 来实现并运行该系统的重复性运营要素，比如排期、路由和报告汇编，让它们按计划自动发生，无需你参与。





第 6 章

# 规模化阶段

# 规模化阶段

在规模化阶段，创始人的角色从构建者重新聚焦为面向公众的高管。产品仍是核心，但你个人的日常工作越来越多地关于公司本身。你的注意力必须扩展到新的规模化阶段活动，比如分析师简报和 IPO 路演，即便你仍努力维持那种精简、以 AI 为中心的结构优势。

## 规模化阶段目标

扩张技术基础设施的工作继续进行，如今又加上了扩张组织本身、并让它成熟为一门生意的工作。在规模化阶段，你面对的是从数千用户到数百万用户、从一个市场到多个市场。在之前每个阶段，增长都是你可以靠贴近用户、并根据紧密反馈回路的数据加上一剂健康的创始人直觉来调整方向、摸着前进的东西。然而现在，目标是构建由成熟组织运营所支撑的系统性增长。对一家 AI 原生创业公司而言，你的目标应是通过累积的深度构建一条可防御的护城河，它源自你构建进产品里的专长、你产品与用户依赖的其他工具和平台的集成深度，以及专有的系统数据和工作流。那些一直朝一个方向、在一致的基础设施上持续构建的创始人，如今拥有了真正难以复制的东西。在这个阶段，公众投资人、分析师、监管者、企业采购团队和收购方会施加更大的压力——以及更大的怀疑——因为现在风险更高。你的产品和组织必须经得起外部审视：不只是你所构建之物的能力，还有围绕它的

治理、合规姿态、财务控制和战略叙事。

## 规模化阶段退出标准

规模化阶段的退出条件不再是单一里程碑，而是一个阈值事件：即便创始人越来越不直接运行日常运营，公司依然可持续。你已展示出系统性增长；构建了能让最苛刻的外部审查者满意的组织治理与合规基础设施；并对“如果一家资金雄厚的在位者今天复制了你的产品，你的用户会留下吗？”这个问题有一个扎实的答案。实践中，这个阈值通常呈现三种形式之一：在不再需要外部资本的规模上可持续盈利、IPO 就绪，或被收购。三者都要求你的增长是系统性且可审计的、你的产品护城河经得起审视，且你的组织在运营上成熟且可持续。当这一切为真时，该祝贺你了：你的创业公司已从一场赌注变成了一门生意。

## 规模化阶段的挑战

### 委派运营层

挑战：规模化阶段的运营系统必须可靠、可持续地运转，而无需有人时刻照看。对一位从第一天起就亲力亲为的创始人来说，这个转变既是结构性挑战，也同样是心理挑战。

你在发布阶段的工作是创建这些系统；在规模化阶段，它变成 (1) 让这些系统成熟到完全可信，并 (2) 然后真正去信任它们。这比听起来更难。即便你是一位善于委派的创始人，也并不总是清楚什么该交出去、什么该留在自己盘子里。交出太多、太快——尤其是交给 AI 自动化系统——关键决策就会在缺少只有创始人才能提供的关键上下文的情况下被做出。然而抓得太久，你又会成为瓶颈。这里的根本挑战，是识别那些只活在创始人脑中或未记录 workflow 里的机构知识，然后把它编纂进有记录、可审计、可转移的系统。

## 扩张技术运营

挑战：客户不再只评估你的产品；他们想知道你的组织能否成为一个可靠的基础设施伙伴。创业前三个阶段的技术挑战都围绕代码库：构建正确的方案而不积累技术债，然后为真实用户加固安全与合规。到了规模化阶段，挑战如今变成围绕代码库的一切；创建支持基础设施、文档，以及标志成熟度的可靠性保证。签订多年合同的更大规模客户和机构买家，在签约前想要这些，并且签约后也会以此要求你。然而，让你走到这一步的同一套 AI 基础设施，能帮你构建有明确响应时间的专门支持职能，以及新客户工程团队真正用得上的文档。

## 扩张组织职能

挑战：一家规模化阶段的公司通常需要招聘、薪酬、会计和法务运营等组织基础设施，无论有多少人在运营它。

在发布阶段，把运营系统化意味着自动化那些消耗创始人注意力的工作流。一家规模化阶段的创业公司，如今需要发展一组更广泛、在某些方面也更具后果的运营职能，举几个例子：财务报告、合规监控、合同管理和客户支持。

## 构建一个 GTM 职能

挑战：有机增长有天花板，而大多数规模化阶段的创始人在不得不构建一个真正的市场进入职能之前就撞上了它。构想、MVP 和发布阶段的增长，常源自创始人主导的销售，从一篇时机恰当的 Product Hunt 帖子，到与早期客户的私人关系。然而这样的有机增长只在某个程度内奏效，而大多数创业公司在规模化阶段撞上这个上限。迹象包括用户曲线趋平、客户获取成本上升，以及只有创始人亲自介入才会推进的管道。规模化阶段的增长，需要构建一台专门的增长引擎，去触达你产品更新、更广的受众。然而，大多数创业创始人，可能从未不得不运营市场营销、销售和分析师关系项目之类的事。一个像样的 GTM 动作不仅需要建立新的系统和流程，还要为“你想如何谈论你的产品”创造一种品牌声音和故事。因为，在创业生命周期的这个阶段，你将需要它去触达的不只是单个新用户，还有投资人和企业买家这样的整个目标受众。幸运的是，GTM 职能不必很大也能有效，而构建出产品的同一套 AI 基础设施，可以引导把它推向市场。

## Claude 如何帮助规模化阶段的创始人

早期创业阶段把 Claude 当作产品本身的基础设施：验证创意的研究伙伴、设计并构建原型的工程团队，以及让单创始人创业成为可能的 AI 运营层。抵达规模化

阶段的 AI 原生创业创始人，如今可以用 Claude、Claude Code 和 Claude Cowork，以他们当初构建的同样方式继续扩张。

## 把日常任务交给 Claude Cowork

以清醒的眼光开启规模化阶段，看清你现在最需要在哪里投入时间和注意力——对从未建过生意的首次创始人来说，这可能是个挑战。Claude 能帮忙，构建一份“此阶段只应由你来做的事”的清单，可能包括产品叙事决策、董事会关系、企业交易，以及创始人对创始人的对话之类。不在那份清单上的任何东西，都是委派或 Claude Cowork 自动化的候选。

- 练习：用 Claude 产出你当前运营层的一张瓶颈地图：当前经由你路由的每一个工作流、决策和审批。现在，请 Claude 推演当你有一周不在时，每一项会发生什么。停滞的那些工作流，正是你仍亲力亲为到足以阻碍进展的地方。这些如何对应到你与 Claude 一起做的创始人优先级与职责清单？接下来，是时候压力测试：你已经构建的系统，是否真的准备好随业务增长而扩张。

- 练习：用 Claude 绘制你当前的工作流，然后问它当你有一周不在时每一项会发生什么。停滞的那些工作流，正是交接标准、升级路径或异常处理仍需收紧的地方。Claude 能帮助分析失败点并推荐恰当的修复，以便你按需更新或替换 Claude Cowork 自动化。

## 把技术运营扩张为企业级基础设施

随着你扩张，买家需要确信你的产品和你的组织可以作为长期基础设施被信任。代码库内部的技术工作一如既往地继续，但现在还有围绕代码库的技术工作要处理。

第一步是把机构知识转化为一个可扩展的系统。用 Claude 起草并维护企业采购期望看到的书面基础设施，包括产品文档、支持手册和 SLA。与此并行，指挥 Claude Code 针对企业合同所要求的特定可靠性和安全标准审计并加固代码库，并搭建基于 Discord 的社区支持从不需要提供的技术支持基础设施：日志、监控、事件响应工具，以及让 SLA 真正可执行的观测性层。然后 Claude Cowork 运行企业支持本身的运营层：工单路由、升级工作流、由产品变更触发的文档更新、续约追踪，以及企业客户成功所依赖的报告节奏。三者合在一起，给一个小团队以一家大得多的组织的支持姿态，而这恰恰是签订一份多年企业合同要求你展示的。

- 练习：挑出你最苛刻的三个潜在客户，或确定三个你梦寐以求想签下的理想客户。请 Claude 产出一份缺口分析：在签订一份多年合同前，这些客户的企业采购团队会期望看到什么样的文档、SLA 和支持基础设施，而你目前在哪里有差距？用这份输出把横跨 Claude Code 和 Claude Cowork 的技术与文档工作排序。

## 构建一个真正的 GTM 职能

创始人的拼劲让你走到这一步，但扩张你的创业公司需要创建并实施一套真正的市场进入策略。AI 能帮你构建、然后运行那台完整的 GTM 引擎。Claude 能协助从零构建基础性 GTM 资源：市场细分、信息架构、分析师关系策略、销售手册，以及一旦你开始与公众投资人、企业买家和华尔街分析师交谈就重要起来的、面向投资人的指标叙事。这些受众各有自己的词汇，并以自己的标准评估你；

Claude 的工作是把你产品的价值主张翻译成对每个受众细分都相关的产品营销方式。如今，Claude Cowork 能成为你的战术执行层：内容管线、外联序列、分析师简报事务、新闻编辑室与公关节奏、CRM 卫生、管道报告，以及把 GTM 策略变成实际商业动作的众多重复周期。在 GTM 动作需要产品营销基础设施的地方——交互式演示环境、集成文档、沙箱租户、API 参考、技术单页——Claude Code 能为你构建它。买家期望从技术上评估你的产品，而在规模化阶段，一个 Loom 视频和一份销售演示已经不够了。这也是让你的 GTM 动作能异步运转的基础设施：一个构建良好的演示环境，能在你开董事会时成交。把领域专长和机构知识转化为 AI 上下文：许多超精简创业创始人，正在为他们在某个特定行业亲身经历或一手观察到的真实问题，构建高度具体的应用或工具。智能体 AI 如今让从未写过一行代码的创始人，也能用他们的领域专长去构建解决复杂问题的产品。Claude、Claude Code 和 Claude Cowork 各自在“把创始人知识转化为复利的产品具体性”中扮演一个角色。用 Claude 来捕捉、组织和提炼创始人知识，把领域专长放到产品够得着的地方。通过延展的对话、项目和记忆，创始人可以把他们知道的一切——行业行话、监管陷阱、边界情况、痛点，以及为何这个问题的显而易见答案行不通的原因——分享进一个结构化、可搜索的上下文里。技能随后能把重复性工作流（例如“我如何审计一份商业租约”“我如何分诊一份病人接诊表”）编纂成 Claude 每次都以同样方式运行的可复用例程。数月之后，这就成为一种没有任何通用 AI 能匹敌的专有知识基底。用 Claude 把你的领域知识外化，对把行业特有的边界情况编码进你的产品变得无比宝贵：例如，一个通用 AI

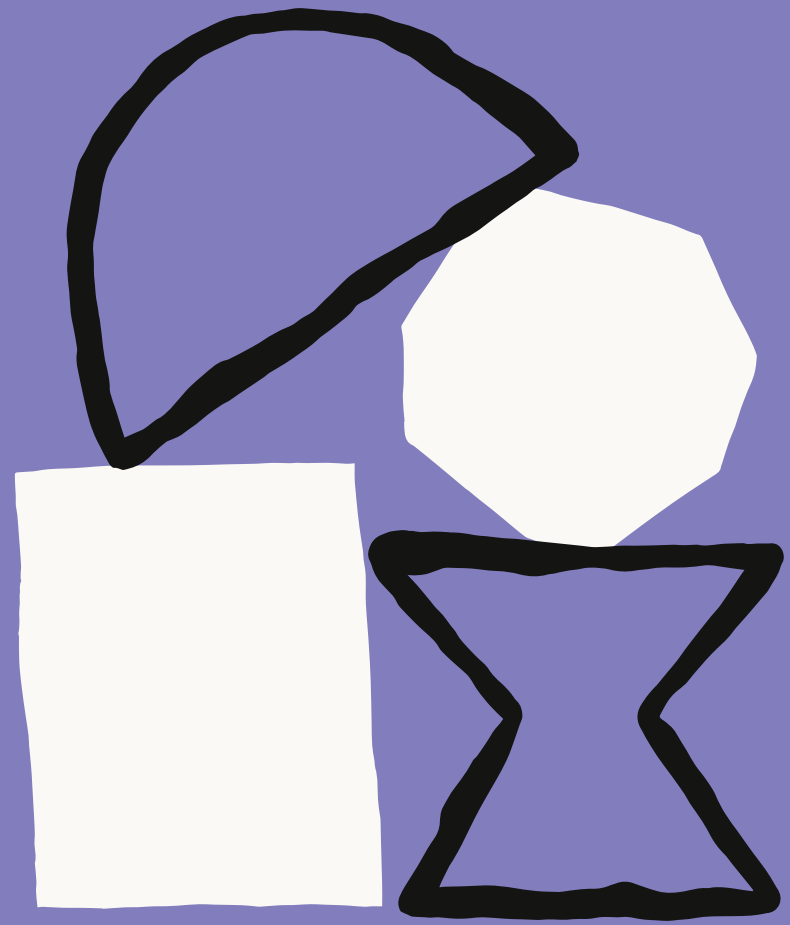
医疗账单工具在 340B 药品计划报销上会崩，而你的工具有专门处理它的逻辑。Claude Code 帮你把你领域里其他专业人士经历的常见痛点，转化为校验逻辑、提示词优化，或与你竞争对手闻所未闻的小众行业系统的 MCP 集成。结果是，你应用或工具的深度和广度都持续复利，方式是竞争对手根本无法复制的。

- 练习：在你的垂直领域里，找出一个通用竞争对手肯定会搞错的边界情况。与 Claude Code 一起，基于一个你确实见过的场景，为它构建一个专门的测试用例（不是单元测试）。每当一个类似的边界情况浮现，就把它加进去。你的测试套件成为你护城河的一张地图。把累积的用户数据复利成可防御的优势：随着用户与你的产品交互，他们产生行为信号（即他们接受哪些输出、拒绝哪些），这为产品路线图提供依据。随着时间推移，你会了解你这一特定用户群体的具体模式、偏好和边界情况。这就是我们所说的复利价值：每一次改进都让产品更有用，从而驱动更多使用，创造更多反馈，驱动更多改进。这些数据被时间锁定、特定于上下文，且模仿者无法重建：你根本买不到数千名一直在你产品里打磨其工作流的用户的行为指纹。Claude 能帮你审计你已收集的任何用户交互数据、识别其中信号最强的行为模式，并设计把持续使用变成系统性模型改进的反馈回路。

- 练习：把你产品交互数据的一份摘要喂给 Claude：你一直在收集什么、收集了多久，以及你对用户随时间如何与产品互动的了解。请它找出该数据中信号最强的三个行为模式，并为每一个设计一条把它变成系统性模型改进的反馈回路。然后请它帮你起草一页纸的护城河叙事，用以指导产品营销：你的数据飞轮如何运转、它已转动了多久，以及为何一个今天起步、资源充足的竞争对手在两年内无法复制它的故事。

创造工作流锁定：复利的数据网络效应让你的产品更难复制，而用户工作流锁定让你的产品更难离开。用户在其日常运营中运行你产品的时间越长，它就越深地嵌入他们实际工作的方式中。他们在它之上构建了自动化、训练人员使用它，并把它连接到他们的数据源和其他工具。他们开发的提示词、打磨的工作流、标准化的输出，全都围绕你产品做什么、怎么做而成形。到这个点，切换就从一个产品决策变成一个全规模运营项目。创造工作流锁定的第一步，是请 Claude 按集成深度绘制你当前的客户群。对每个客户细分，识别他们在你产品之上构建了哪些工作流、依赖哪些集成。这显示出你产品在哪儿有黏性、在哪儿需要更深入。你提供的集成越多，客户就有越多的表面积去构建依赖你产品的工作流。Claude Code 帮你快速搭建与你目标用户依赖的数据管线、项目管理工具和其他系统的原生集成。Claude Code 还能构建让客户不只是使用你产品、而是在它之上构建的 API、webhook 和 SDK——这是最深形式的锁定。

- 练习：请 Claude 帮你为你的前十大客户构建一次工作流集成审计。对每一个，记录他们构建的自动化、依赖的集成、贯穿你产品的团队工作流，以及你对他们切换成本的估计。然后请 Claude 识别这一组里的模式：哪些类型的集成为你这个具体产品创造最深的锁定，以及你能构建或启用什么，来为当前还停留在表层的客户加深集成。



第 7 章

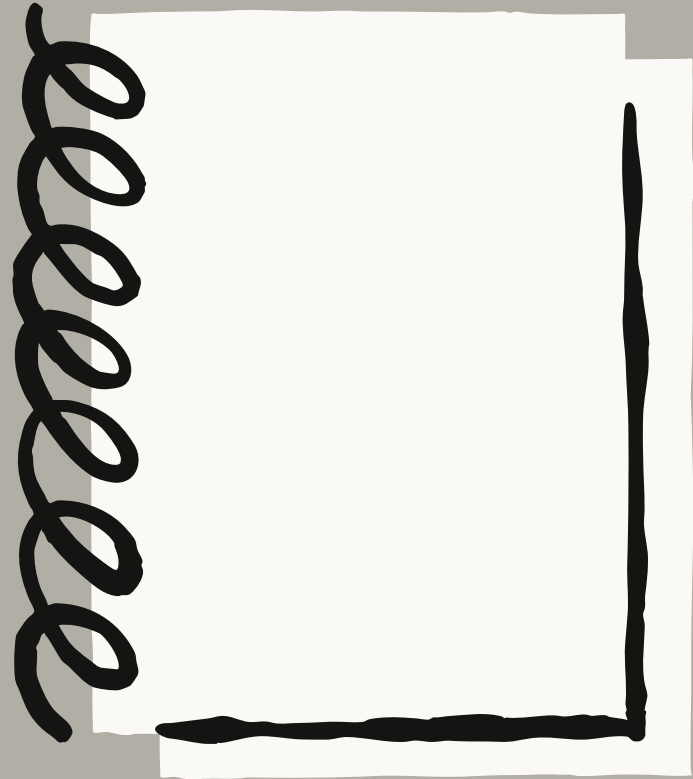
# 工作没变，规则已新

# 工作没变，规则已新

在 AI 时代，创始人的工作没有变：找到一个真实的问题，构建解决它的东西，并把它扩张成一家重要的公司。变的是抵达那里的路径。横跨四个阶段——构想、MVP、发布与规模化——AI 把数个季度压缩成数周。过去要花数月的验证周期，现在只需几个下午。一个可运行的原型不再需要一位具备合适技术栈的联合创始人；它需要一个清晰的问题，以及与一个编码智能体的几次专注会话。发布就绪从发布前的手忙脚乱压缩成一条持续的工作流。而在规模化阶段，过去迫使早期招聘陷入救火角色的运营重负，如今越来越能交给 AI，把你的团队解放出来，把注意力花在那些成为你护城河的判断决策上。瓶颈不再是你能构建什么，而是你选择构建什么。



资源



# 资源

## 用 Claude 构建

- Building AI Agents for Startups：分享创业公司如何用智能体，随着扩张变得更少依赖创始人。
- Claude Code 文档：把构建者从初始安装带到高级智能体工作流。小贴士：从“How Claude Code works”概览开始。
- Claude Code 最佳实践：涵盖在 Anthropic 内部以及各工程团队中行之有效的模式——上下文管理、权限、规划与验证工作流。
- 使用 CLAUDE.md 文件：逐步讲解如何为你特定的代码库配置 Claude Code。对设置开发环境的 MVP 阶段创始人而言是必读。
- Claude Code 高级用户技巧：突出来自 Claude Code 团队本身的工作流模式，包括并行会话和验证回路。
- 开始使用 Claude Cowork：分享团队如何设置 Claude Cowork，并开始实施技能、插件等功能，跨你的创业公司放大其影响。
- 教程：claude.com/resources/tutorials 提供一份可搜索的、针对具体任务的实操演练清单。

## 创始人故事

- 三家 VC 创业公司如何用 Claude Code 构建他们的公司：考察 HumanLayer (F24)、Ambral (W25) 和 Vulcan Technologies (S25) 如何用 Claude 把原型快速推向市场，并以智能体编码工作流扩张 AI 驱动的平台。
- GC AI 的创始人运用领域专长，构建了一个响应迅速、由 Claude 驱动的法律平台，贴合内部团队实际的工作方式：公司特定的手册、跨职能干系人，以及可变的风险容忍阈值。

- Carta Healthcare 用 Claude 为其临床抽取平台提供动力，每年处理 22,000 例外科病例，并把数据抽取时间减少 66%。

- Anything，由 Claude 和 Agent SDK 驱动，已帮助 150 万用户在不写代码的情况下把想法变成可运行的软件产品，包括一位非技术创始人构建并已在销售一个完整的招聘平台。Anything 的 AI 智能体处理整个构建，让个体创业者得以专注于他们的领域专长。

- Cogent 是一家应用 AI 实验室，构建智能体以自动化关键的企业安全任务。该创业公司用 Claude 作为智能体的推理层，在整个漏洞生命周期内自动化调查、优先级排序和整改。

- Airtree 把 Claude Cowork 作为其运营基础设施的中心，统一过去散落在十几个不同工具和团队中的数据。现在，当一个人用技能构建一个工作流自动化时，组织里每个人都能用它去完成待办清单上那些从未做完的事。

- Duvo 构建 AI 智能体，跨 ERP、供应商门户、电子表格、电子邮件，甚至电话，运行采购、供应链和品类管理流程。Duvo 完全构建在 Claude 之上，用 Agent SDK 跨工作流编排。

- Zingage 是一个为家庭护理机构 7x24 自动化运营而构建的 AI 智能体平台。该创业公司用 Claude 的结构化工具调用，跨一个 EMR 和多个沟通渠道编排，并用 Claude 的情境推理构建能给出细致、因患者而异结果的智能体，而非对最常见回应做模式匹配。

- Kindora 是一个由一位非营利高管构建的 AI 驱动平台，他用 Claude Sonnet 构建了一个亟需的工具，用以智能匹配慈善机构与资助方。在把数千个匹配筛选到

少数值得追求的之后，Kindora 的 MCP 连接器让非营利组织直接在 Claude 内访问其勘探工具。

- Wordsmith 由一位律师转 CTO 创立，为内部法务团队提供可靠的 AI 驱动法律技术。Claude 是 Wordsmith 合同审查、协议起草和文档审查能力的推理引擎，该创业公司的工程团队用 Claude Code 构建并演进平台本身。

## 创业支持与机会

- Anthropic 创业计划：面向与 Anthropic 风投合作伙伴合作的创业公司，该计划提供免费 API 额度、公开可得的最高一档速率限制，以及受邀参加专属创始人活动和工作坊。
- Claude 社区：面向构建者的论坛和社区空间。
- 实时学习资源：会议、网络研讨会、直播和录像。

